

Artificial Intelligence

Assignment 1

State-Space Representation of the 8-Puzzle Problem

1. Problem Statement

The 8-Puzzle is a classic search problem consisting of a 3×3 board containing eight numbered tiles (1 to 8) and one blank space. The tiles are arranged in a scrambled order on the board. The blank space can be interchanged with any tile that is physically adjacent to it (up, down, left, or right), effectively “moving” that tile into the blank position.

The objective is to transform a given initial (scrambled) arrangement of tiles into a specified goal arrangement using the smallest possible number of moves, by repeatedly sliding tiles into the blank space. Since the number of possible board configurations is finite but large ($9!/2 = 181,440$ reachable states), the problem is well suited to being represented and solved as a state-space search problem, where each unique arrangement of the board is treated as a “state” and each legal tile-slide is treated as an “operator” that transitions from one state to another.

2. Objectives

1. To represent the 8-Puzzle problem formally as a state-space search problem.
2. To define the initial state, goal state, and a suitable state representation for the puzzle.
3. To identify the set of legal operators/actions that transform one state into another.
4. To construct a state-space diagram illustrating how states are generated and connected through operators.
5. To study how search strategies (e.g., BFS, DFS, A*) can be applied over this state space to find a solution path.
6. To explore real-world applications of state-space search techniques.

3. Initial State

The initial state is the starting arrangement of the 8 tiles and the blank space on the 3×3 board, as given in the problem instance. For illustration, consider the following initial configuration (0 represents the blank space):

1	2	3
	4	6
7	5	8

Fig. 1 – Example Initial State (S_0)

4. Goal State

The goal state is the target arrangement that the puzzle must reach, typically the tiles arranged in ascending order with the blank space in a fixed position (commonly the last cell):

1	2	3
4	5	6

7	8	
---	---	--

Fig. 2 – Standard Goal State (Sg)

5. State Representation

Each state of the puzzle is represented as a 3×3 matrix (or an equivalent 9-element array/tuple) holding the digits 1 through 8 and 0 (for the blank), where each digit's position denotes the tile occupying that cell of the board.

Example (row-major array form of the initial state above):

State $S_0 = [1, 2, 3, 0, 4, 6, 7, 5, 8]$

This representation is compact, uniquely identifies every configuration of the board, and makes it straightforward to check goal-matching, generate successor states, and store visited states (to avoid revisiting them) during search.

6. Operators/Actions

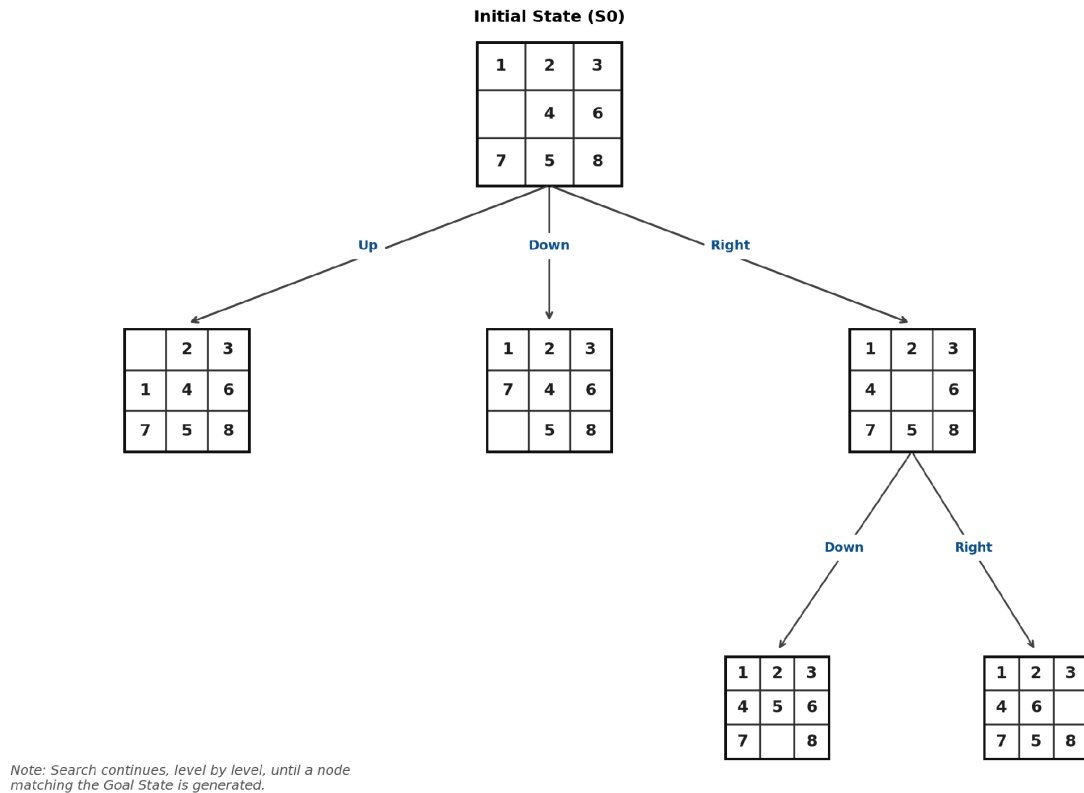
An operator moves the blank tile in one of four directions, provided the move stays within the board boundaries. Moving the blank in a direction is equivalent to sliding the adjacent numbered tile in the opposite direction.

- Up – Move the blank tile one position upward (slide the tile above it down).
- Down – Move the blank tile one position downward (slide the tile below it up).
- Left – Move the blank tile one position to the left (slide the tile to its left, right).
- Right – Move the blank tile one position to the right (slide the tile to its right, left).

At most four operators are applicable at any state; corner cells allow only two moves, edge cells allow three, and the center cell allows all four. Each operator has a uniform cost of 1, so the total path cost equals the number of tile moves.

7. State Space Diagram

The state-space diagram below shows the initial state as the root node. Applying each applicable operator generates child (successor) states, forming a tree/graph structure. The search continues, level by level, expanding nodes until a state matching the goal state is found.



Note: Search continues, level by level, until a node matching the Goal State is generated.

Figure: Partial State Space Diagram for the 8-Puzzle Problem

8. Applications

State-space search, as illustrated by the 8-Puzzle problem, underlies many practical AI systems:

- Robotics and Path Planning – Representing a robot's possible positions/configurations as states and planning a sequence of moves from a start position to a target position while avoiding obstacles.
- Puzzle and Game Solvers – Solving sliding-tile puzzles (15-puzzle), Rubik's Cube, Sudoku, and similar combinatorial puzzles/games using search algorithms.
- Route/Navigation Systems (GPS Navigation) – Modelling road networks as states (locations) and operators (road segments) to find the shortest or fastest route between a source and a destination.
- Automated Planning Systems – Planning task sequences in logistics, scheduling, and manufacturing, where each state represents a configuration of resources/tasks and operators represent allowable actions.
- Puzzle-based Cognitive Training and AI Education – Used as a benchmark problem to teach and evaluate search algorithms such as BFS, DFS, IDA*, and A*.

9. References

1. Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson Education.
2. Rich, E., Knight, K., & Nair, S. B. (2009). Artificial Intelligence (3rd ed.). Tata McGraw-Hill.
3. Luger, G. F. (2008). Artificial Intelligence: Structures and Strategies for Complex Problem Solving (6th ed.). Pearson Education.
4. GeeksforGeeks. (n.d.). 8 puzzle Problem using Branch and Bound. Retrieved from <https://www.geeksforgeeks.org/>

5. Course lecture notes and slides on State-Space Search, Artificial Intelligence course.